

[例 7.2] “高校本科教务管理”信息系统的视图集成。

图 7.11、图 7.22(b)、图 7.26 分别给出了“学生学籍管理”“学生选课管理”和“教师教学管理”子系统的分 E-R 图。图 7.30 为“高校本科教务管理”信息系统的基本 E-R 图(这里基本 E-R 图中各实体的属性因篇幅有限从略)。

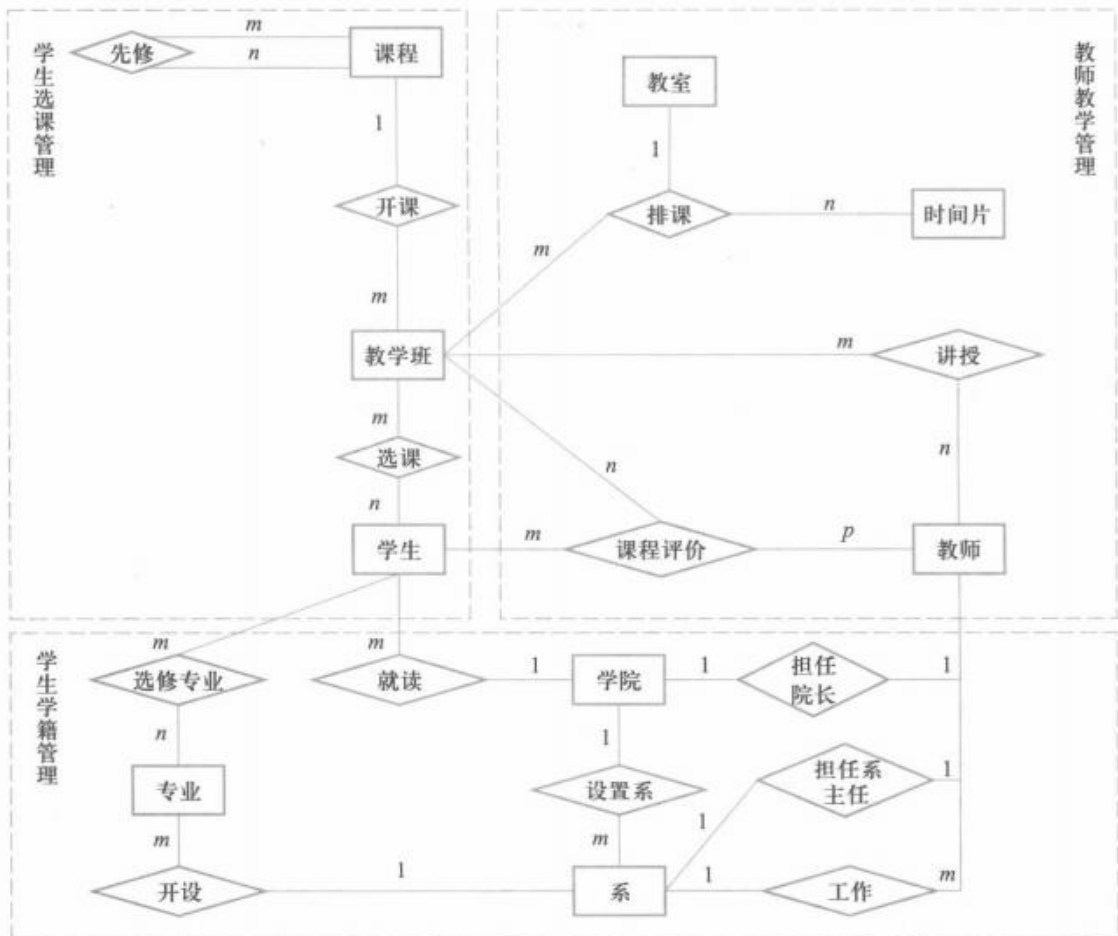


图 7.30 “高校本科教务管理”信息系统的基本 E-R 图

7.4 逻辑结构设计

概念结构是独立于任何一种数据模型的信息结构，逻辑结构设计任务就是把概念结构设计阶段得到的基本 E-R 图转换为与选用数据库管理系统产品所支持的数据模型相符合的逻辑结构。

目前的数据库应用系统普遍采用关系数据库管理系统，所以这里只介绍 E-R 图向关系数据库模型的转换原则与方法。

7.4.1 E-R 图向关系模型的转换

E-R 图向关系模型的转换要解决的问题是,如何将实体型和实体间的联系转换为关系模式,以及如何确定这些关系模式的属性和码。

关系模型的逻辑结构是一组关系模式的集合。E-R 图则是由实体型、实体的属性和实体型之间的联系三个要素组成的,所以将 E-R 图转换为关系模型实际上就是要将实体型、实体的属性和实体型之间的联系转换为关系模式。下面介绍转换的一般原则。一个实体型转换为一个关系模式,关系的属性就是实体的属性,关系的码就是实体的码。

对于实体型间的联系有以下不同的情况:

① 一个一对一联系可以转换为一个独立的关系模式,也可以与任意一端对应的关系模式合并。如果转换为一个独立的关系模式,则与该联系相连的各实体的码以及联系本身的属性均转换为关系的属性,每个实体的码均是该关系的候选码。如果与某一端实体对应的关系模式合并,则需要在该关系模式的属性中加入另一个关系模式的码和联系本身的属性。

② 一个一对多联系可以转换为一个独立的关系模式,也可以与 n 端对应的关系模式合并。如果转换为一个独立的关系模式,则与该联系相连的各实体的码以及联系本身的属性均转换为关系的属性,而关系的码为 n 端实体的码。

③ 一个多对多联系可以转换为一个关系模式,与该联系相连的各实体的码以及联系本身的属性均转换为关系的属性,各实体的码组成关系的码或关系码的一部分。

④ 三个或三个以上实体间的一个多元联系可以转换为一个关系模式。与该多元联系相连的各实体的码以及联系本身的属性均转换为关系的属性,各实体的码组成关系的码或关系码的一部分。

⑤ 具有相同码的关系模式可合并。

[例 7.3]把图 7.30 中“学生学籍管理”子系统的分 E-R 图转换为关系模型。关系的码用下划线标出。

① 学院(学院编号,学院名,建院时间,院长)

此为“学院”实体型对应的关系模式。该关系模式已包含了联系“担任院长”所对应的关系模式。“学院编号”属性是“学院”关系模式的主码,“院长”属性参照关系模式“教师”的主码“职工号”。

② 系(系编号,系名,联系人,联系方式,系主任,所在学院)

此为“系”实体型对应的关系模式。该关系模式已包含了联系“担任系主任”和联系“设置系”所对应的关系模式。“系编号”属性是该关系模式的主码,“系主任”属性参照关系模式“教师”的主码“职工号”,“所在学院”属性参照关系模式“学院”的主码“学院编号”。

③ 专业(专业编码,专业名,类别,年限,开设系)

此为“专业”实体型对应的关系模式。该关系模式已包含了联系“开设”所对应的关系模式。“专业编码”属性是该关系模式的主码,“开设系”属性参照关系模式“系”的主码“系编号”。

④ 教师(职工号, 姓名, 职称, 出生日期, 所在系)

此为“教师”实体型对应的关系模式。该关系模式已包含了联系“工作”所对应的关系模式。“职工号”属性是该关系模式的主码,“所在系”属性参照关系模式“系”的主码“系编号”。

⑤ 学生(学号, 姓名, 性别, 出生日期, 所在学院)

此为“学生”实体型对应的关系模式。该关系模式已包含了联系“就读”所对应的关系模式。“学号”属性是该关系模式的主码,“所在学院”属性参照关系模式“学院”的主码“学院编号”。

⑥ 选修专业(学号, 专业编码, 是否主修)

此为联系“选修专业”所对应的关系模式。“学号”和“专业编码”属性构成了该关系模式的主码,“学号”参照关系模式“学生”的主码“学号”,“专业编码”参照关系模式“专业”的主码“专业编码”。

这些关系的示例元组,可参考附录“高校本科教务管理”信息系统的 E-R 图和关系模式。

7.4.2 数据模型的优化

数据库逻辑设计的结果不是唯一的。为了进一步提高数据库应用系统的性能,还应该根据应用需要适当地修改、调整数据模型的结构,这就是数据模型的优化。

关系数据模型的优化通常以规范化理论为指导,其方法如下:

① 确定数据依赖。在 7.2.3 小节中已讲到用数据依赖的概念分析和表示数据项之间的联系,写出每个数据项之间的数据依赖。按需求分析阶段所得到的语义,分别写出每个关系模式内部各属性之间的数据依赖以及不同关系模式属性之间的数据依赖。

② 对于各个关系模式之间的数据依赖进行极小化处理,消除冗余的联系,具体方法已在 7.3.5 小节中介绍过。

③ 按照数据依赖的理论对关系模式逐一进行分析,考察是否存在部分函数依赖、传递函数依赖、多值依赖等,确定各关系模式分别属于第几范式。

④ 根据需求分析阶段得到的处理要求,分析对于这样的应用环境这些模式是否合适,确定是否要对某些模式进行合并或分解。

必须要注意的是,并不是规范化程度越高的关系就越优。例如,当查询涉及两个或多个关系模式的属性时,系统经常要进行连接运算。连接运算的代价是相当高的,可以说关系模型低效的主要原因就在于连接运算。这时可以考虑将这几个关系合并为一个关系。因此在这种情况下,第二范式甚至第一范式也许是合适的。

又如,非 BCNF 的关系模式虽然从理论上分析会存在不同程度的更新异常或冗余,但如果在实际应用中对此关系模式只是查询,并不执行更新操作,则不会产生实际影响。所以对于具体应用来说,到底规范化到什么程度需要权衡响应时间和潜在问题两者的利弊来决定。

⑤ 对关系模式进行必要分解,提高数据操作效率和存储空间利用率。常用的两种分解方法是水平分解和垂直分解。

水平分解是把(基本)关系的元组分为若干子集合,定义每个子集合为一个子关系,以提高系统的效率。根据“80/20 原则”,一个大关系中经常被使用的数据只是关系的一部分,约 20%,则可以把经常使用的数据分解出来,形成一个子关系。如果关系 R 上具有 n 个事务,而且多数事务存取的数据不相交,则 R 可分解为少于或等于 n 个子关系,使每个事务存取的数据都对应一个关系。

垂直分解是把关系模式 R 的属性分解为若干子集合,形成若干子关系模式。垂直分解的原则是将经常在一起使用的属性从 R 中分解出来形成一个子关系模式。垂直分解可以提高某些事务的效率,但也可能使另一些事务不得不执行连接操作,从而降低了效率。因此是否进行垂直分解取决于分解后 R 上所有事务的总效率是否得到了提高。垂直分解需要保证分解后的关系具有无损连接性和保持函数依赖性。可以用第 6 章中介绍的模式分解算法对需要分解的关系模式进行分解和检查。

规范化理论为数据库设计人员判断关系模式的优劣提供了理论标准,可用来预测模式可能出现的问题,使数据库设计工作有了严格的理论基础。

7.4.3 设计用户外模式

将概念模型转换为全局逻辑模型后,还应该根据局部应用需求,结合具体关系数据库管理系统的特点设计用户的外模式。

目前关系数据库管理系统一般都提供了视图概念,可以利用这一功能来设计更符合局部用户需要的用户外模式。

定义数据库全局模式主要是从系统的时间效率、空间效率、易维护等角度出发。由于用户外模式与模式是相对独立的,因此在定义用户外模式时可以注重考虑用户的习惯与方便。具体包括以下几方面:

1. 使用更符合用户习惯的别名

在合并各分 E-R 图时曾做过消除命名冲突的工作,以使数据库系统中同一关系和属性具有唯一的名字。这在设计数据库整体结构时是非常必要的。用视图机制可以在设计用户视图时重新定义某些属性名,使其与用户习惯一致,以方便使用。

2. 对不同级别的用户定义不同的视图

可以对不同级别的用户定义不同的视图,以保证系统的安全性。例如,针对“教师教学管理”子系统内的教师、学生、教学班三者之间的教学评价关系,为教师建立教师-教学班评价视图(职工号、教师.姓名、教学班号、课程号、课程名、评价内容、评价类型、教师反馈),为学生建立学生-教学班-评价视图(学号、学生.姓名、教师.姓名、教学班号、课程号、课程名、评价内容、评价类型、教师反馈)。

教师-教学班评价视图只包含教师对应教学班的所有课程评价信息,隐藏了提供教学评价

意见的学生信息,从而保证了学生信息的安全性;而学生-教学班-评价视图不仅可以看到该学生自己对所有课程的评价信息,也能够看到教师的反馈意见。基于教师-教学班评价视图,并把该视图授权给对应的教师用户,教师只能查询本人被评价的记录,不能看到提供教学评价意见的学生信息,这样的设计就可以使学生能认真评价教师的教学效果,减少不必要的担心。

3. 简化用户对系统的使用

如果某些局部应用中经常要使用某些很复杂的查询,为了方便用户,可以将这些复杂查询定义为视图,用户每次只对定义好的视图进行查询,大大简化了用户的使用。

注意:因为扩展 E-R 模型是选读部分,本章略去了扩展 E-R 图的集成以及向关系模型的转换。

7.5 物理结构设计

数据库在物理设备上的存储结构与存取方法称为数据库的物理结构,它依赖于选定的数据库管理系统。为一个给定的逻辑数据模型选取一个适合应用要求的物理结构的过程,就是数据库的物理结构设计。

数据库的物理结构设计通常分为两步:

- ① 确定数据库的物理结构。在关系数据库中主要指存取方法和存储结构。
- ② 对物理结构进行评价。评价的重点是时间和空间效率。

如果评价结果满足原设计要求,则可进入物理结构实施阶段,否则就需要重新设计或修改物理结构,有时甚至要返回逻辑设计阶段修改数据模型。

7.5.1 数据库物理结构设计的内容和方法

不同的数据库产品所提供的物理环境、存取方法和存储结构有很大差别,能供设计人员使用的设计变量、参数范围也很不相同,因此没有通用的物理结构设计方法可遵循,只能给出一般的设计内容和原则,希望设计优化的数据库物理结构,使得在数据库上运行的各种事务响应时间小、存储空间利用率高、事务吞吐量大。为此,首先对要运行的事务进行详细分析,获得选择数据库物理结构设计所需要的参数,其次要充分了解所用关系数据库管理系统的内部特征,特别是系统提供的存取方法和存储结构。

对于数据库查询事务,需要得到如下信息:

- 查询的关系。
- 查询条件所涉及的属性。
- 连接条件所涉及的属性。
- 查询的投影属性。

对于数据更新事务,需要得到如下信息:

- 被更新的关系。